

Definitional Equalities for Substitutions using Rewrite Rules

Anonymous Author(s)

Abstract

Choosing a representation of variable binding is a central problem in mechanizing programming language metatheory. De Bruijn indices provide a canonical representation, but developments built on them require numerous lemmas about how substitutions compose and how they pass under binders, and every one of those lemmas has to be applied by hand or by a tactic.

The σ -calculus makes parallel substitutions first-class and normalizes them by rewriting. Modeling substitutions as functions turns its rules into theorems, which is what libraries such as AUTOSUBST exploit, but the resulting equalities stay propositional.

We register the σ -calculus as rewrite rules in Agda, so that substitutions are normalized during conversion rather than by a tactic. Two requirements pull against each other. The rules are provable only if the operations compute, and eligible for rewriting only if the involved definitions are stuck.

We identify which σ -calculus version is eligible for rewriting, and derive a rewrite system where renamings stay first-class, because proof assistants need them to define substitution by structural recursion. We supply a code generator that emits the whole development from a syntax description. Every rule is registered together with a proof of the equality it installs, so consistency is preserved. To test the rules at scale we solve the POPLmark and POPLmark Reloaded challenges.

Keywords: Explicit Substitutions, Term Rewrite Systems, Agda

1 Introduction

Handling variable binding remains a central challenge in mechanizing programming language metatheory. Compared to pen-and-paper proofs, binders in proof assistants are a notorious source of uninteresting technicalities and boilerplate. Many representations have been proposed to contain them, among them locally nameless binders [11], nominal syntax [37, 38], higher-order abstract syntax [27], and de Bruijn indices [16]. De Bruijn syntax makes α -equivalent terms syntactically identical, and the syntax itself is easy to define. A development built on de Bruijn indices still pays for a long list of lemmas about substitution composition and about the interaction of substitution with binders.

On paper, substitution usually replaces a single variable. In a proof assistant one instead works with parallel substitutions, which map variables to terms and replace all of them simultaneously. The σ -calculus [1] makes such maps first-class objects with primitive syntactic operations for identity, weakening, extension and composition, together with a rewrite system that normalizes expressions built from them. The de Bruijn algebra [31] is the canonical semantic model for those operations, a two-sorted algebra over de Bruijn terms and substitutions, with substitutions modeled as total functions from variables to terms. In that model the rules are theorems, which is what makes them usable for equational reasoning in proof assistants. AUTOSUBST [32] and its successor AUTOSUBST 2 [36] generate the primitive operations, the proofs of the laws, and a tactic that normalizes substitutions inside a goal.

The equalities obtained by AUTOSUBST are propositional. Normalization becomes a step the user takes, so every goal that mentions a substitution has to be brought into normal form before it can be closed. In Rocq that step is close to automatic, because AUTOSUBST supplies a tactic that performs the whole normalization procedure, but what the tactic produces is still a propositional equality. Recent support for user-defined rewrite rules in Agda [13, 14] and Rocq [23] removes the step. A proof of a propositional equality can be registered as a rule, and its left-hand side then reduces to its right-hand side, so the equality holds definitionally. The difference can be large for intrinsically typed syntax, when substitutions appears in a type index and every use of a propositional law becomes a transport [9]. Consistency of rewrite rules is retained as long as the registered rules are supported by proofs and the resulting system is confluent and terminating [14]. Agda applies a rewrite rule by matching a stuck term against the left-hand side of the registered proof [2]. A term is stuck when it is headed by a defined function, a postulate or a constructor, and not reducing.

Whether the left-hand side of a substitution law from the σ -calculus is eligible for rewriting is decided by how the operations in it are defined. The same choice decides whether that law is provable. Modeled as functions from variables to terms or as vectors of terms, the σ -calculus operations compute, and every law is provable. That same computation affects the left-hand sides of rewrite rules, which leaves them not stuck and therefore not registrable. If we decide to model the operations as constructors, every left-hand side is stuck by construction, but we cannot derive proofs for the laws, because a constructor has nothing to compute. Wadler takes a minimal version of the second route, making only weakening

CPP '27,

2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/XXXXXX.XXXXXX>

a constructor in the syntax, thereby successfully avoiding renamings [39]. But weakening then does not push inwards into terms, and the definitional equalities that would push it there are unrecoverable.

We choose a definition using functions whose laws are provable and block their reduction artificially, when needed. Agda's opaque definitions [18] give the required control over the reduction behavior of definitions. A function declared opaque does not reduce outside its block, and an unfolding instruction lets a later block see through it again. Hence we can see the operations as ordinary functions where the proofs need them and as stuck definitions where the rules need to match them.

Opacity settles how the operations are defined, and leaves open which σ -calculus variant can be registered as rewrite rules. To make the matter more complicated, we also want renamings to be first class, because a proof assistant needs them to define substitution by structural recursion.

Contributions

- **A σ -calculus.** We give an σ -calculus variant with first-class renamings, that we can make definitional in a multi-sorted, intrinsically scoped setting using Agda. Every rule is proved before it is registered, so logical consistency is preserved.
- **A metatheoretic study of the rewrite system.** Agda's built-in checker can validate local confluence, and using an external checker we can prove termination and confluence of the first-order export.
- **A code generator.** We supply a script that emits the entire infrastructure from a higher-order abstract syntax description.
- **An evaluation of the approach.** We solve the POPLmark Challenge, Parts 1A, 1B, 2A, 2B and 3, and POPLmark Reloaded, on definitions and the rewrite system generated by the script. Only one fact remains propositional: a certain substitutions is a renaming in disguise.

Structure

In section 2 we explain the scoped syntax setting we work in and define renaming and substitution over it. In section 3 we state the conditions Agda puts on a rewrite rule, identify the σ -calculus variant that meets them, derive the rules that follow with first class renamings, and establish confluence and termination. In section 4 we exercise the system on the metatheory of System F, explain how to generate the infrastructure from a syntax description, and run it at scale on the POPLmark challenge. Section 5 discusses related work, section 6 collects future work, and section 7 concludes.

2 Syntax and Substitution

The development is carried out in Agda throughout. Proofs in Agda are significantly easier when terms are definitionally equal, that is, when they reduce to the same term. When they do not, users must resort to manual equational reasoning with propositional equality, which is cumbersome as Agda lacks good support for automated proof-discharge tactics. Maximizing definitional equalities thus directly reduces manual effort.

This section only establishes the required definitions. We give the multi-sorted scoped syntax of the paper (section 2.1) and define renaming and substitution over it (section 2.2). Two features of these definitions are important for what section 3 can register as rewrite rules. Variables form a sort of their own, generated by zero and successor, as is standard and required to represent all variables. Renamings are defined separately, because substitution is not structurally recursive without them.

2.1 Multi-Sorted and Scoped Syntax

Many programming languages feature several syntactic categories. For instance, System F [17, 28] has three: expressions, types, and kinds. Each usually gets its own datatype definition. Figure 1 (left) shows the standard intrinsically scoped implementation, where types are indexed by the number of type variables and expressions by the numbers of type and expression variables. Variables are correct-by-construction de Bruijn indices of type `Fin n`, the natural numbers smaller than n . Thus, scoped syntax rules out scoping errors without committing to full intrinsic typing.

The standard representation forces one substitution operation per pair of sorts, such as type-in-type, type-in-expression and expression-in-expression. This is the representation AUTOSUBST 2 chooses, and it explains why their rewrite system needs to be extended with vector substitutions, which bundle the substitutions over the different sorts and come with normalization rules of their own.

To avoid this combinatorial explosion we unify the syntax along two orthogonal dimensions, shown in fig. 1 (right). Variables and terms form a single datatype $S \vdash [m] s$, indexed by a `Mode` m , a target `Sort` s and a `Scope` S , a list of sorts. Variables of type $S \ni s$ are represented across all sorts at once, the length of S being the number of variables in scope and the position of s in S the de Bruijn index. Renamings and substitutions defined over this syntax then operate uniformly across sorts, and vector substitutions become unnecessary.

Recent work extends it to the maps over that syntax [6, 25], deriving renaming and substitution from a single traversal that abstracts over what a variable is mapped to, a variable or a term. Our syntax admits the same treatment, and some laws of section 3 would then follow uniformly for renamings and substitutions rather than twice in similar shape. We keep

```

221 data Kind : Set where
222   ★ : Kind
223
224 data Type (n : Nat) : Set where
225   ' _ : Fin n → Type n
226   ∀[α:_]_ : Kind → Type (suc n) → Type n
227   _⇒_ : Type n → Type n → Type n
228
229 data Expr (n m : Nat) : Set where
230   ' _ : Fin m → Expr n m
231   λx_ : Expr n (suc m) → Expr n m
232   Λα_ : Expr (suc n) m → Expr n m
233   ' _ : Expr n m → Expr n m → Expr n m
234   ' _ : Expr n m → Type n → Expr n m
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

Figure 1. System F as classically scoped syntax (left) and as the multi-sorted scoped syntax we use (right)

renaming and substitution apart for clarity of presentation, and because several of our rules treat the two differently.

2.2 Renaming and Substitution

Instantiation, the operation that applies a substitution to a term, when defined by mutual recursion with composition is not structurally recursive. As is standard, we follow Adams [3] and define instantiation with renamings first. Renamings then run through the whole development, and proofs about them need the same reasoning we want to automate for substitutions. We will later follow AUTOSUBST 2 and put both into the equational theory and relate the two, when possible.

We model renamings and substitutions as functions (fig. 2). A renaming ($_ \rightarrow^R _$) takes every variable to a variable of the same sort, changing only the scope. A substitution ($_ \rightarrow^S _$) takes it to a term of that sort instead. We call either of them a *map* where the distinction does not matter. A map then has a mode as well, the mode of its images, **V** for a renaming and **T** for a substitution. The operations over maps are functions as well, and all of them are placed in an **opaque** block. They stay stuck, and we unfold them only in the proofs that need their computational behavior.

Modeling maps as functions is a choice. A vector of images, one per variable in the domain scope, would do as well, and would need the same opacity. Extension then is a constructor and stuck by itself, but every other operation is still a definition that unfolds. We take functions because we find them easier to think about, at the price of postulating

function extensionality. We return to the vector model in section 4.2.

The identity (id^R) takes every variable to itself, and takes its sort argument explicitly because Agda frequently fails to infer it. Weakening (wk^R) takes a variable to its successor. Extension ($_ \rightarrow^R _$) adds an image for a newly bound variable, and lifting ($_ \uparrow^R _$) fixes that variable and weakens the images of the rest, which is what lets a renaming pass under a binder.

Instantiation ($_ \rightarrow^R _$) traverses a term and lifts whenever it enters a binder. It is defined on variables as well, so that a lookup is written $x \llbracket \xi \rrbracket^R$ rather than as the application $\xi _ x$. We only show three of its clauses, for a variable, a lambda binding and an application, and omits the five that are evident congruences. Composition ($_ \circ^R _$) looks a variable up in the first renaming and renames the result with the second, hence it's only ordinary function composition in disguise.

The coercion operator ($_ \rightarrow _$) embeds a renaming in a substitution, wrapping each image variable in the variable constructor. The remaining substitution operations are the same one level up, with terms in place of variables, and the figure lists them in that order. Extension ($_ \rightarrow^S _$) reads as before. Lifting ($_ \uparrow^S _$) weakens the images of the rest with the weakening renaming, which is what keeps the recursion structural. Instantiation ($_ \rightarrow^S _$) and composition ($_ \circ^S _$) read as before as well, except that a lookup now returns a term. The identity (id^S) and weakening (wk^S) substitutions are non-opaquely defined in terms of the coercion operator.

$\rightarrow^R : \text{Scope} \rightarrow \text{Scope} \rightarrow \text{Set}$
 $S_1 \rightarrow^R S_2 = \forall s \rightarrow S_1 \ni s \rightarrow S_2 \ni s$

opaque

$\text{id}^R : S \rightarrow^R S$

$\text{id}^R _ x = x$

$\text{wk}^R : \forall s' \rightarrow S \rightarrow^R (s' :: S)$

$\text{wk}^R _ x = \text{succ } x$

$_ \cdot^R : S_2 \ni s \rightarrow S_1 \rightarrow^R S_2 \rightarrow (s :: S_1) \rightarrow^R S_2$

$(x \cdot^R \xi) _ \text{zero} = x$

$(_ \cdot^R \xi) _ (\text{succ } x) = \xi _ x$

$_ \uparrow^R : S_1 \rightarrow^R S_2 \rightarrow \forall s \rightarrow (s :: S_1) \rightarrow^R (s :: S_2)$

$(\xi \uparrow^R _) _ \text{zero} = \text{zero}$

$(\xi \uparrow^R _) _ (\text{succ } x) = \text{succ } (\xi _ x)$

$_ \llbracket _ \rrbracket^R : S_1 \vdash [m] s \rightarrow S_1 \rightarrow^R S_2 \rightarrow S_2 \vdash [m] s$

$_ \llbracket _ \rrbracket^R \{m = \mathbf{V}\} x \xi = \xi _ x$

$(_ x) \llbracket \xi \rrbracket^R = _ (x \llbracket \xi \rrbracket^R)$

$(\lambda x e) \llbracket \xi \rrbracket^R = \lambda x (e \llbracket (\xi \uparrow^R _) \rrbracket^R)$

$(e_1 \cdot e_2) \llbracket \xi \rrbracket^R = (e_1 \llbracket \xi \rrbracket^R) \cdot (e_2 \llbracket \xi \rrbracket^R)$

-- ...

$_ \circ^R : S_1 \rightarrow^R S_2 \rightarrow S_2 \rightarrow^R S_3 \rightarrow S_1 \rightarrow^R S_3$

$(\xi_1 \circ^R \xi_2) _ x = (\xi_1 _ x) \llbracket \xi_2 \rrbracket^R$

$\rightarrow^S : \text{Scope} \rightarrow \text{Scope} \rightarrow \text{Set}$
 $S_1 \rightarrow^S S_2 = \forall s \rightarrow S_1 \ni s \rightarrow S_2 \vdash s$

opaque

$\langle _ \rangle : S_1 \rightarrow^R S_2 \rightarrow S_1 \rightarrow^S S_2$

$\langle \xi \rangle _ x = _ (x \llbracket \xi \rrbracket^R)$

$_ \cdot^S : S_2 \vdash s \rightarrow S_1 \rightarrow^S S_2 \rightarrow (s :: S_1) \rightarrow^S S_2$

$(t \cdot^S \sigma) _ \text{zero} = t$

$(t \cdot^S \sigma) _ (\text{succ } x) = \sigma _ x$

$_ \uparrow^S : S_1 \rightarrow^S S_2 \rightarrow \forall s \rightarrow (s :: S_1) \rightarrow^S (s :: S_2)$

$(\sigma \uparrow^S _) _ \text{zero} = _ \text{zero}$

$(\sigma \uparrow^S _) _ (\text{succ } x) = (\sigma _ x) \llbracket \text{wk}^R _ \rrbracket^R$

$_ \llbracket _ \rrbracket^S : S_1 \vdash [m] s \rightarrow S_1 \rightarrow^S S_2 \rightarrow S_2 \vdash s$

$_ \llbracket _ \rrbracket^S \{m = \mathbf{V}\} x \sigma = \sigma _ x$

$(_ x) \llbracket \sigma \rrbracket^S = \sigma _ x$

$(\lambda x e) \llbracket \sigma \rrbracket^S = \lambda x (e \llbracket (\sigma \uparrow^S _) \rrbracket^S)$

$(e_1 \cdot e_2) \llbracket \sigma \rrbracket^S = (e_1 \llbracket \sigma \rrbracket^S) \cdot (e_2 \llbracket \sigma \rrbracket^S)$

-- ...

$_ \circ^S : S_1 \rightarrow^S S_2 \rightarrow S_2 \rightarrow^S S_3 \rightarrow S_1 \rightarrow^S S_3$

$(\sigma_1 \circ^S \sigma_2) _ x = (\sigma_1 _ x) \llbracket \sigma_2 \rrbracket^S$

$\text{id}^S : S \rightarrow^S S$

$\text{id}^S = \langle \text{id}^R \rangle$

$\text{wk}^S : \forall s' \rightarrow S \rightarrow^S (s' :: S)$

$\text{wk}^S s' = \langle \text{wk}^R s' \rangle$

Figure 2. The operations for renaming (left) and for substitution (right)

3 Rewrite Rules for Renamings and Substitutions

What we need of a σ -calculus is a finite and confluent set of rules, and not every variant provides one for the definitions we gave in section 2. We state the conditions Agda puts on a rewrite rule in section 3.1, identify a variant that meets them in section 3.2, derive the rewrite system that follows with first class renamings in section 3.3, and establish confluence and termination in section 3.5. Throughout, the multi-sorted scoped System F is the running example, but the construction applies to any multi-sorted scoped syntax.

3.1 Rewrite Rules in Agda

User-defined rewrite rules [13, 14] extend Agda's normalization-by-evaluation approach for definitional equalities by treating propositional equalities as reduction rules. A proof of $f p_1 \dots p_n \equiv v$ is eligible for rewriting under three conditions.

- f is a postulate, a defined function or a constructor.

- The left-hand side is stuck on f , that is, the application does not reduce.
- Every variable bound by the rule occurs on the left-hand side, and occurs there in pattern position.

The last condition serves matching. Applying a rule means matching its left-hand side against a stuck term, and that match has to determine every variable the rule binds. Matching is higher-order [2], that is, a variable bound by the rule may stand for a function and occur applied on the left-hand side. In general that problem is undecidable, and the requirement that all bound variables need to appear in pattern position is the restriction that makes it decidable. A bound variable appears in pattern position if it is either not applied at all, or applied to distinct bound variables and to nothing else.

For example, renaming a term by the identity leaves it unchanged. The law is proved by induction on the term, so it does not hold definitionally on its own:


```

441 [id] : ∀ {n} (t : Tm n) → t [ idR ] ≡ t
442 [id] (‘ x) = refl
443 [id] (t1 · t2) = cong2 _·_ ([id] t1) ([id] t2)
444 -- ...
445

```

It meets the three conditions: the left-hand side is stuck on the defined function `_[]`, because no clause matches a term using a variable pattern, and its only bound variable t occurs there unapplied. Once it is registered via `{-# REWRITE [id] #-}`, Agda treats $t [id^R]$ as definitionally equal to t :

```

452 _ : ∀ {n} {t : Tm n} → t [ idR ] ≡ t
453 _ = refl
454

```

Beyond the syntactic restriction, all rewritten equations as a whole have to be well behaved, in two respects. Subject reduction needs confluence of all registered rewrite rules with respect to the already existing reductions, and consistency is preserved when all rules are instantiated with propositional equality proofs [14, Sect. 3 and 6.4]

Confluence is decided on the critical pairs of the rule set. The terms to which two reductions apply, need to reduce to a common term. A rule has to be joinable against every rule already registered, and against the reduction Agda performs on its own, hence a rule interacts with the defining clauses of every function it mentions.

Section 3.3 proves every rule before registering it, and section 3.5 establishes confluence.

3.2 Chooseing the Correct Variant

The σ -calculus exists in several variants, and they differ on properties the registration as rewrite rules depends on. A $\lambda\sigma$ -calculus additionally includes the β -rules of the object language, which in our setting belong to the user's language description and lie outside the theory we register. Our rules deal only with substitutions (and renamings later). For the purpose of this subsection id^S and wk^S are constants of the calculus. Section 2.2 defines them through the coercion operator, and that definition plays no part until renamings enter in section 3.3 for the purpose of termination.

All variants share a set of rules that goes back to the original calculus of Abadi et al. [1], where lifting is defined in terms of the other operators: $\sigma \uparrow^S s = (‘ zero) \cdot^S (\sigma \circ^S wk^S s)$. We group them into four categories and name them after Stark et al. [36].

- Reduction laws push an instantiation through term constructors. There is one for every constructor of the syntax, as in

$$(\lambda x e) [\sigma]^S = \lambda x (e [\sigma \uparrow^S s]^S)$$

They are exactly the clauses of the instantiation operator of section 2.2.

- The defining clauses of the operations evaluate a substitution at a variable, so they fix the meaning of extension, weakening and lifting, for example

$$zero [t \cdot^S \sigma]^S = t$$

- Interference laws govern the interplay of composition, extension and weakening, which include associativity of composition and distributivity of composition over extension:

$$(\sigma_1 \circ^S \sigma_2) \circ^S \sigma_3 = \sigma_1 \circ^S (\sigma_2 \circ^S \sigma_3)$$

$$(t \cdot^S \sigma_1) \circ^S \sigma_2 = (t [\sigma_2]^S) \cdot^S (\sigma_1 \circ^S \sigma_2)$$

- The monad laws relate instantiation to composition. Identity states $t [id^S]^S = t$, and compositionality fuses two instantiations into one,

$$(t [\sigma_1]^S) [\sigma_2]^S = t [\sigma_1 \circ^S \sigma_2]^S$$

In our model, where substitutions are functions, the reduction laws and the defining clauses of the operations hold by computation. The monad laws are proved by induction on the term and the interference laws are proved using function extensionality.

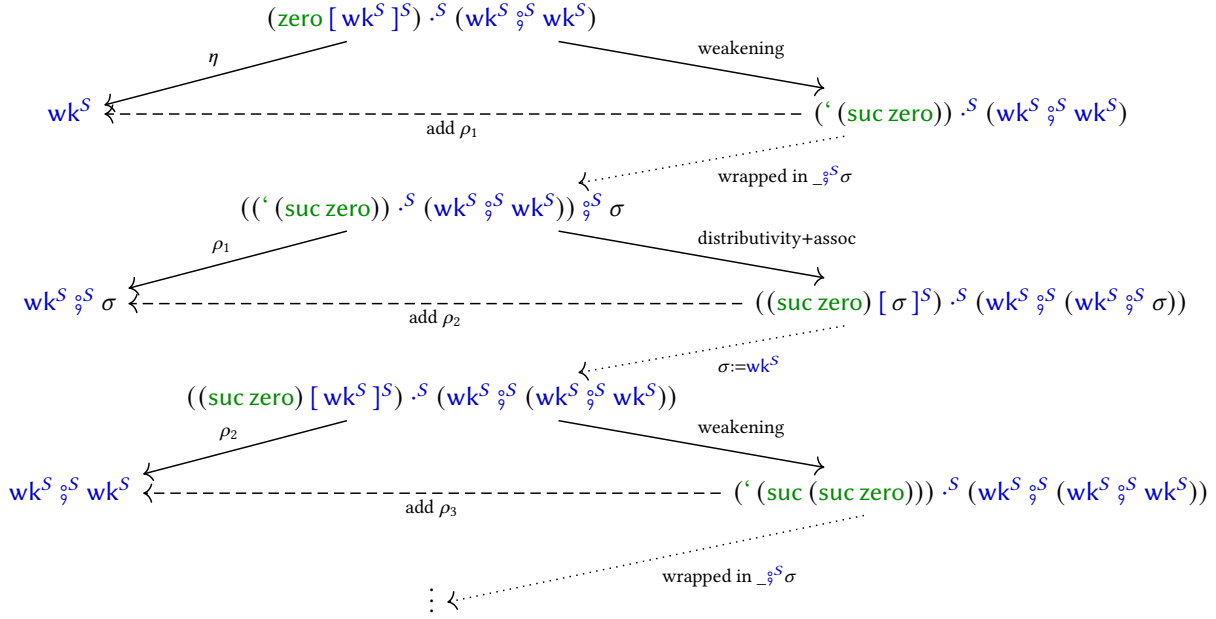
The original σ -calculus establishes confluence on ground terms only. That does not suffice for a rewrite system inside a proof assistant, because a rule applies wherever the type checker finds a match and the terms it encounters contain metavariables.

σ_{SP} [31, 35] repairs this by adding new rules. The η -law equates a substitution with its value at the first variable extended by its restriction to the remaining ones: $(zero [\sigma]^S) \cdot^S (wk^S s \circ^S \sigma) = \sigma$. They can be seen as surjective pairing for substitutions. With the η -laws the calculus is even complete for a model in which substitutions are also functions from variables to terms. Every equation that holds is derivable from the rules [31, 35].

The downside is, that the η -laws clash with inductive variables. The left-hand side of η mentions σ twice, under a lookup and under a composition, so every rule that evaluates either of the two overlaps η .

Inductive variables force the law $x [wk^S s]^S = ‘(suc x)$. With the variables generated by `zero` and `suc`, the weakened variable and `suc x` are two terms for the same variable, and pattern matching on variables needs the constructor form to be the normal form.

Now instantiate σ in η with a weakening, which opens the pair in the top row of fig. 3. No rule collapses the extension on its right, because the only rules matching an extension are the η -laws and both need the head to be `zero`. Joining it means adding the rule ρ_1 of the figure, and that rule has an extension on its left-hand side. Distributivity fires on an extension that is the left operand of a composition, so ρ_1 and distributivity overlap and the middle row is their pair. The rule ρ_2 that joins that one carries a lookup in the head, so instantiating σ with a weakening evaluates the lookup and

Figure 3. Attempt to complete the η -law against inductive variables

reaches the bottom row, which is the top row with one more **suc**.

Observation 1. Alongside inductive variables the η -laws leave non-joinable critical pairs that cannot be finitely completed.

σ_{SP} has no **suc**. The n -th variable is represented as **zero** under n weakenings, so a lookup at a weakening is already a normal form, and oriented as rewrite rules using **postulate** its equations together with both η -laws pass the local confluence check.

If we wish to eliminate the postulates to guarantee consistency, we need weakening inside the term syntax, which is exactly the work of Wadler [39]. But then we have the issue that weakening does not push into terms. Hence we would never be able to instantiate all laws we require with proofs, to preserve consistency.

AUTOSUBST can combine inductive variables with the η -laws because a tactic applies its equations where the user directs them using tactics. Confluence on metavariables is not a critical property for tactics.

$\sigma_{\uparrow}$ treats lifting as a primitive operation and is confluent on open terms [15]. Its variables are generated inductively, and a weakened variable reduces to its successor, which is what we require. The downside is that the η -laws hold only propositionally, and thus the calculus is not complete. We forfeit completeness in any case because we need renamings as a second kind of map, and the transfer between renamings and substitutions is incomplete. Hardin et al. [19]

tabulate the 23 rules of $\sigma_{\uparrow}$, which fall into the four categories above together with a family of rules under a composition. Composition is associative, and associativity normalizes it to the right, so a rule that matches a composition, such as the interference law interact, $wk^S ;^S (t .^S \sigma) = \sigma$, does not fire when its redex sits inside a right-nested composition, $wk^S ;^S ((t .^S \sigma) ;^S \tau)$. Completion modulo an associative operator repairs this by restating each such rule under a composition [26], here $wk^S ;^S ((t .^S \sigma) ;^S \tau) = \sigma ;^S \tau$. One level suffices, because associativity turns any deeper nesting $((t .^S \sigma) ;^S \tau) ;^S \tau'$ back into $(t .^S \sigma) ;^S (\tau ;^S \tau')$, which the restated rule matches.

None of these variants provides renamings, which are required for termination in proof assistants. The standard repair is to define renamings first [3]. AUTOSUBST 2 takes the further step of making renamings first-class and extending the equational theory with rules that relate renamings and substitutions [36]. Their extended system is no longer complete, and its confluence was not formally verified [35].

3.3 Extension with Renamings

We make renamings first-class in $\sigma_{\uparrow}$. The rules include two copies of $\sigma_{\uparrow}$, one for maps of each mode, with the reduction laws instantiated to the constructors of the signature. The remaining rules connect the two modes. This section only presents the connection and the rules where renamings and substitutions need to be treated differently.

Renamings and substitutions are connected using the coercion operator $\langle _ \rangle$. Lifting a substitution weakens the images

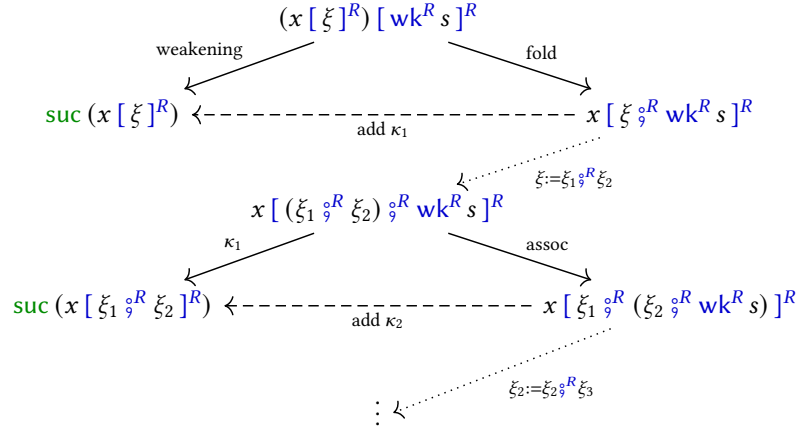


Figure 4. Attempt to complete the fold for renamings against weakening

of the rest with the weakening renaming, which is what keeps the recursion structural, and id^S and wk^S are the coerced renamings $\langle \text{id}^R \rangle$ and $\langle \text{wk}^R s \rangle$. Due to that we need to make sure substitutions that are coerced renamings reduce to actual renamings for the appropriate rules to apply. For instance, the coincidence law [35] is oriented that way,

$$t[\langle \xi \rangle]^S = t[\xi]^R$$

A term instantiated with an embedded renaming is renamed directly. The remaining coercion rules keep embedded renamings closed under the substitution operations by moving $\langle _ \rangle$ outward, as in $\langle \xi_1 \rangle \circ \langle \xi_2 \rangle = \langle \xi_1 \circ \xi_2 \rangle$, which we call $\langle _ \rangle$ -comp, and $(\langle \xi \rangle \uparrow^S s) = \langle \xi \uparrow^R s \rangle$.

Compositionality exists in the four combinations of the two modes, each at terms and at variables, and each of the eight rules is registered in one of two directions. We call the direction that merges two instantiations into one instantiation under a composed map a fold, and the direction that takes a composed map apart into two applications a push.

The eight compositionality rules point as follows:

$$\begin{aligned} (t[\xi_1]^R)[\xi_2]^R &= t[\xi_1 \circ \xi_2]^R && \text{fold} \\ x[\xi_1 \circ \xi_2]^R &= (x[\xi_1]^R)[\xi_2]^R && \text{push} \\ (t[\xi]^R)[\sigma]^S &= t[\langle \xi \rangle \circ \sigma]^S && \text{fold} \\ x[\langle \xi \rangle \circ \sigma]^S &= (x[\xi]^R)[\sigma]^S && \text{push} \\ (t[\sigma]^S)[\xi]^R &= t[\sigma \circ \langle \xi \rangle]^S && \text{fold} \\ (x[\sigma]^S)[\xi]^R &= x[\sigma \circ \langle \xi \rangle]^S && \text{fold} \\ (t[\sigma_1]^S)[\sigma_2]^S &= t[\sigma_1 \circ \sigma_2]^S && \text{fold} \\ (x[\sigma_1]^S)[\sigma_2]^S &= x[\sigma_1 \circ \sigma_2]^S && \text{fold} \end{aligned}$$

The direction depends only on the first map. When we have a renaming on the left, the term case folds and the variable case pushes. When we have a substitution, then we always fold.

At variables the push is forced. Because weakening needs to be related to the successor, $(x[\xi]^R)[\text{wk}^R s]^R$ reduces to $\text{succ}(x[\xi]^R)$. Under a fold at variables it also reduces to $x[\xi \circ \text{wk}^R s]^R$, and the only sound join is a rule matching $\xi \circ \text{wk}^R s$, a composition whose constructor is the right operand, which conflict the right associativity. Note that left associativity would have a similar issue. Figure 4 follows the attempt to complete the fold, where associativity moves the weakening one composition deeper at every step.

Observation 2. Compositionality laws for variables with renamings on the left are registered as push, instead of a fold.

The push is mirroring the definition of composition. This is not a coincidence, because we want to apply the definition for composition, exactly when we can later apply a defining law for renamings. For compositionality with two renamings this might be intuitive, but for a renaming composed with a substitution this might not be obvious from the start.

3.4 The Full Rewrite Sytem

The rest of the rules deal with the completion of rewrite system that resulted from our choices in the last two sections.

Table 1 counts the resulting system by group. Only the reduction laws depend on the signature. We use $|c|$ to denote the amount of constructors the syntax has. Every other group is the same for any syntax. The 4 groups above the middle line originate from the σ_Π -calculus, and are copied once for renamings and once for substitutions. The other three groups complete the rewrite system and connect the maps of the two modes.

The η -laws stay theorems we cannot include in the rewrite system. We also cannot register distributivity in the renaming copy of the σ_Π for renamings, because it overlaps with

Table 1. The registered rules by group, counted once per mode. Three of them are stated for both modes at once, which is why the development registers 72 rules and the export of section 3.5 has 75.

#	group	content
2 * c	reduction laws	push an instantiation through constructors, one rule per constructor and mode
9	defining clauses	evaluate a map at a variable, the meaning of extension, weakening and lifting
17	interference laws	the interplay of composition, extension, weakening and lifting for both renamings and substitutions
10	monad laws	identity and compositionality, in all four mode combinations
9	under a composition	restate rules whose redex sits inside a composition
5	push rules	completion rules for the push strategy
9	coercion	rules that collapse coercions into renamings

associativity and only a fold would join the two, which would be against our push-at-V strategy.

The full rule set is available in the supplement. Because we have modeled maps as functions, all rules are proven propositionally and then registered as rewrite rule.

3.5 Correctness

What remains is confluence. To this end, we rely on Agda's opt-in `--local-confluence-check`, which computes the critical pairs of the rewrite system, reports those it cannot join, and certifies that every one of ours does. It says nothing about termination, which Agda does not check for rewrite rules at all. That obligation therefore has to leave Agda, and it cannot be skipped, because a non-terminating rule set makes the type checker loop only on a term that triggers the divergence and may otherwise survive undetected. We export the system as a first-order term rewriting system, with scope indices erased, and hand the resulting 75 rules to external provers. AProVE proves termination in 23.9 s and confluence in 24.9 s. Local confluence implies confluence only for a terminating system, so the two together give confluence by Newman's lemma. The two results agree by independent routes, and the obligations of section 3.1 are discharged.

The system does *not* pass Agda's global `--confluence-check`, which reports 277 errors, every one of them `RewriteAmbiguousRules` and none a non-joinable pair. That check demands more than confluence. The most general unifier of any two overlapping left-hand sides has to be a registered left-hand side as well, and every rule has to satisfy the triangle property. No finite rule set containing an associative operator meets the first condition. A single associativity rule over a postulated binary operator already fails it, although that rule terminates and Agda itself certifies it locally confluent. It overlaps itself

at a threefold composition, and registering that instance creates an overlap at a fourfold one. We measured what completion costs on our own system. Adding the 23 rules the checker asks for first takes the error count from 277 to 529, and what it asks for next is the same family one composition deeper. Every rule it asks for is true and provable in two lines, so what defeats the completion is their number. The condition is a closure property, and our system is confluent regardless, by the local check and Newman's lemma above.

Both checks certify the rule set as it is seen from outside the `opaque` blocks, where the operations do not reduce. An `unfolding` block puts the defining clauses back into the reduction relation, and neither check covers that combination; the proofs of the rules live in such a block, but they are discharged before any rule is registered. The certificate therefore carries an obligation the type checker cannot enforce: once a rule is in scope, its operations must never be unfolded again. `abstract` does not serve here, although it looks like the stronger seal. Agda's confluence checker reduces through it deliberately, so that rewriting cannot break subject reduction inside an abstract block, and the same development declared abstract reports 197 non-joinable critical pairs.

Cost. Definitional equalities are paid for at type-checking time. Measured on one core under Agda 2.8, a module that does nothing but import a generated core costs about 70 s, against 1 s for a comparable module importing only the standard library. The figure is flat in the size of the rule set, 69 s at 77 rules and 66 s at 111. Per-module times across the development run from 74 to 120 s, so that fixed import cost dominates and the proof content is cheap. The Part 3 animation is the exception, at about 76 minutes, and there the rules are doing evaluation instead of conversion.

Two controls make the verdicts meaningful. Both properties hold at two signature sizes, three and eight constructors, which is what licenses the generator claim. And the setup has a falsifier that fires, because adding the two η -laws is refuted for confluence by three independent tools, as section 3.2 predicts. Evidence at two sizes is not a proof schematic in the signature, and obtaining one would mean decomposing the system along the lines of compositional confluence criteria [33].

4 Implementation and Evaluation

We first exercise the rule set on the metatheory of System F (section 4.1). A construction shown on one language is a claim about that language. The rest of this section turns it into a claim about the method. We generate the infrastructure from a syntax description (section 4.2) and measure what it buys on the POPLmark Challenge (section 4.3).

4.1 What the Rules Buy

We exercise the rule set on the standard metatheory of the System F of fig. 1, against a typing judgment `⊢_:_` and a

$\vdash\lambda :$
 $(t ::_t \Gamma) \vdash e : (\text{weaken } t') \rightarrow$
 $\Gamma \vdash (\lambda x e) : (t \Rightarrow t')$
 $\vdash\bullet :$
 $\Gamma \vdash e : (\forall[\alpha: k] t') \rightarrow$
 $\Gamma \vdash t : k \rightarrow$
 $(k ::_t \Gamma) \vdash t' : k' \rightarrow$
 $\Gamma \vdash (e \bullet t) : (t' [t]_0)$
 $\beta\text{-}\lambda :$
 $\text{Val } e_2 \rightarrow$
 $((\lambda x e_1) \cdot e_2) \hookrightarrow (e_1 [e_2]_0)$
 $\text{wk-cancel} : \forall \{t : S \vdash s\} \{t' : S \vdash s'\} \rightarrow (\text{weaken } t) [t']_0 \equiv t$
 $\text{wk-cancel} = \text{refl}$
 $\text{wk-comm} : \forall \{t : S_1 \vdash s\} \{\sigma : S_1 \rightarrow^S S_2\} \rightarrow$
 $(\text{weaken } \{s' = s'\} t) [(\sigma \uparrow^S s')]^S \equiv \text{weaken } (t [\sigma]^S)$
 $\text{wk-comm} = \text{refl}$
 $\text{subst-commute} : \forall \{t : (s' :: S_1) \vdash s\} \{t' : S_1 \vdash s'\}$
 $\{\sigma : S_1 \rightarrow^S S_2\} \rightarrow$
 $(t [(\sigma \uparrow^S s')]^S) [t' [\sigma]^S]_0 \equiv (t [t']_0) [\sigma]^S$
 $\text{subst-commute} = \text{refl}$
 $-- \text{ the induction hypothesis types the body at } (\text{weaken } t') [\sigma \uparrow^S _]^S,$
 $-- \text{ while } \vdash\lambda \text{ demands } \text{weaken } (t' [\sigma]^S). \text{ Discharged by wk-comm.}$
 $\text{sub-pres } \{\sigma = \sigma\} (\vdash\lambda \vdash e) \vdash \sigma = \vdash\lambda (\text{sub-pres } \{\sigma = \sigma \uparrow^S _ \} \vdash e (\vdash\uparrow^S \{\sigma = \sigma\} \vdash \sigma _))$
 $-- \vdash\bullet \text{ concludes at } t' [t]_0, \text{ so the two sides are}$
 $-- (t' [\sigma \uparrow^S _]^S) [t [\sigma]^S]_0 \text{ and } (t' [t]_0) [\sigma]^S.$
 $-- \text{ Discharged by subst-commute.}$
 $\text{sub-pres } \{\sigma = \sigma\} (\vdash\bullet \vdash e \vdash t \vdash t') \vdash \sigma = \vdash\bullet (\text{sub-pres } \{\sigma = \sigma\} \vdash e \vdash \sigma) (\text{sub-pres } \{\sigma = \sigma\} \vdash t \vdash \sigma)$
 $(\text{sub-pres } \{\sigma = \sigma \uparrow^S _ \} \vdash t' (\vdash\uparrow^S \{\sigma = \sigma\} \vdash \sigma _))$
 $-- \vdash\lambda \text{ stores the result type weakened, so the redex is typed at}$
 $-- (\text{weaken } t_2) [e_2]_0 \text{ where the goal is } t_2. \text{ Discharged by wk-cancel.}$
 $\text{sr } (\vdash\cdot \{e_2 = e_2\} (\vdash\lambda \vdash e_1) \vdash e_2) (\beta\text{-}\lambda v_2) =$
 $\text{sub-pres } \{\sigma = e_2 \cdot^S \text{id}^S\} \vdash e_1 (\vdash\uparrow \vdash e_2)$

Figure 5. Everything the three interesting clauses depend on. Top left, the two typing rules and the reduction rule whose types carry a substitution. Top right, the two statements and the notion of a well typed substitution. Middle left, the three laws this creates a demand for, each holding by `refl`; middle right, the two helpers the clauses call. Bottom, the only three clauses of either proof in which a law is spent, with the law named. Every other clause needs none.

call-by-value small-step semantics $\hookrightarrow$. A context `Ctx` S assigns to every variable in scope S a type one sort up, so $\Gamma \vdash e : t$ reads as usual. Figure 5 collects everything the argument below depends on.

Substitution enters this metatheory in exactly three places, shown top left. $\vdash\lambda$ stores the result type of a function *weakened*, because the body is typed in a longer context. $\vdash\bullet$ concludes at a single substitution $t' [t]_0$, because instantiating a polymorphic type substitutes for its bound variable. And $\beta\text{-}\lambda$ contracts a redex to $e_1 [e_2]_0$. Each of the three puts a substitution somewhere a proof will later have to move it, and each therefore creates a demand for one law.

Top right are the two statements. A substitution is well typed, written $\sigma : \Gamma_1 \rightarrow^S \Gamma_2$, when it sends every variable x typed t in Γ_1 to a term $x [\sigma]^S$ typed $t [\sigma]^S$ in Γ_2 . The type is substituted along with the term, which is what makes the notion compose. `sub-pres` then says that applying such a substitution to a well typed term preserves typing, and `sr` is subject reduction. Both are the expected inductions and use two helpers, shown middle right. $\vdash\uparrow^S$ lifts a well typed substitution under a binder, and is proved from the corresponding statement for renamings, which is what keeps the recursion structural. $\vdash\uparrow$ turns a well typed term into the well typed single substitution that the two β -cases need.

Middle left are the three laws the three obligations create a demand for. A de Bruijn development states and applies each of them by hand. Here each holds by **refl**, because each is a normalization rather than a lemma: the rules turn both sides into the same normal form, and **refl** sees that they agree. Written out for subst-commute, the classic substitution lemma, the two branches are

$$\begin{aligned}
 & (t[\sigma \uparrow^S s]^S)[t'[\sigma]^S]_0 = (t[\sigma \uparrow^S s]^S)[t'[\sigma]^S \cdot^S \text{id}^S]^S \\
 \rightarrow & t[(\sigma \uparrow^S s) \circ^S (t'[\sigma]^S \cdot^S \text{id}^S)]^S \\
 \rightarrow & t[t'[\sigma]^S \cdot^S (\sigma \circ^S \text{id}^S)]^S \\
 \rightarrow & t[t'[\sigma]^S \cdot^S \sigma]^S \\
 & (t[t']_0)[\sigma]^S = (t[t' \cdot^S \text{id}^S]^S)[\sigma]^S \\
 \rightarrow & t[(t' \cdot^S \text{id}^S) \circ^S \sigma]^S \\
 \rightarrow & t[t'[\sigma]^S \cdot^S (\text{id}^S \circ^S \sigma)]^S \\
 \rightarrow & t[t'[\sigma]^S \cdot^S \sigma]^S
 \end{aligned}$$

Five rules and no user intervention. Every intermediate term above is checked against the endpoint in the supplement, so the trace is the one Agda takes. A normal form is reached when no traversal can be pushed further, no composition can be fused, and no embedded renaming is left in a substitution. On a closed term the substitution disappears entirely. On an abstract t it survives as a single application to a right-nested, identity-free map, as here.

The bottom of the figure closes the loop, and each clause can now be read against the rule above it. In the $\vdash \lambda$ case the induction hypothesis types the body at $(\text{weaken } t')[\sigma \uparrow^S s]^S$, because that is where $\vdash \lambda$ put it, while $\vdash \lambda$ demands $\text{weaken } (t'[\sigma]^S)$ to rebuild the abstraction. That is wk-comm. In the $\vdash \bullet$ case the goal is $(t'[t]_0)[\sigma]^S$ and the three sub-derivations supply $(t'[\sigma \uparrow^S s]^S)[t[\sigma]^S]_0$, which is subst-commute. In the β -case for $\lambda x_-, \vdash \lambda$ has stored the result type weakened, so **sub-pres** delivers the contractum at $(\text{weaken } t_2)[e_2]_0$ where the goal is plain t_2 , which is wk-cancel.

Every other clause of either proof needs nothing, and for two different reasons. $\vdash \Delta$, $\vdash \cdot$ and $\vdash^*$ conclude at a type the substitution never has to be moved through, so no equation arises. The β -case for $\lambda \alpha_-$ does face one, $t'[t]_0$ against $t'[t \cdot^S \text{id}^S]^S$, but the two are the same term by the definition of single substitution. Because the three that do arise are conversions as well, every clause of both proofs is a bare constructor application.

The three are not special. Two single substitutions can be exchanged, lifting distributes over composition, and the same again for renamings. So do the two mixed laws relating the modes, which a development without first-class renamings cannot state at all. The supplement checks every one of them, and every one is **refl**.

One case does need help, and the cause is the rewriting itself. In the two β -cases we pass σ explicitly, writing **sub-pres** $\{\sigma = e_2 \cdot^S \text{id}^S\}$. By the time Agda sees the goal, the

rules have normalized the type index, so σ can no longer be recovered by unifying against $e[\sigma]^S$. Definitional equality and inferability pull against each other here. The more of the index is normalized before unification, the less of it is left to read a metavariable off. The cost is one implicit argument, and no substitution law is applied by hand.

4.2 Code Generation

Nothing in section 3 is specific to System F. Of the seven groups in table 1, only the reduction laws depend on the signature, and they depend on it in the simplest possible way, two rules per constructor. Everything else is schematic in the constructor list, so the whole construction can be generated. We provide a script that turns a higher-order abstract syntax (HOAS) description into Agda code with the rules ready for rewriting. Figure 6 is the description of our running example, and it is the entire input: eleven constructors across three sorts, with binders written as function arguments. AUTOSUBST 2 does the same for Rocq, emitting the scoped syntax together with the substitution lemmas that need induction over it.

Several of AUTOSUBST 2's features come for free. Vector substitutions are unnecessary and mutually recursive definitions work unchanged, both because the syntax is multi-sorted. Inline Agda code, such as lists or products of terms, can be encoded with additional sorts at the cost of standard library support. Two features are missing. A distinction between sorts with and without binders would take an additional index on **Sort**. Variadic syntax could be encoded through arguments to the sort constructors [29].

Adding a constructor to a language is mechanical, because only the reduction group depends on it and it contributes exactly two rules there. Binders that bind several variables at once are not free in the same way. They are handled by a lifting operation that lifts by a *list* of sorts, and the completion has to be redone at each lifting level, which is what the iterated $\uparrow^*$ family costs. This is why the generated cores carry more rules than the 72 of section 3.3: 77 for STLC, 83 with sums, 87 for $F_{<}$, 101 with records and 111 with pattern matching. Everything above 72 is the $\uparrow^*$ family and the extra reduction laws of the larger signatures.

The script is a single Python file with strict type checking enabled, comprising a parser for HOAS descriptions and ad-hoc code generation, and it carries all boilerplate Agda code so that others can run it standalone. Beside the five languages of section 4.3 we generate a dozen files from the HOAS descriptions shipped with AUTOSUBST 2, which is the cheapest evidence we have that the schema is not tuned to our own examples.

Using Vectors instead of Functions

A vector-like data type is the other candidate representation, with the map a finite list of terms. Extension alone is a constructor. Identity, weakening, lifting and composition

```

1101 kind : Type
1102 type : Type
1103 expr : Type
1104
1105 -- the constructors for kind
1106 star : kind
1107
1108 -- the constructors for type
1109 arr : type -> type -> type
1110 all : kind -> (type -> type) -> type
1111
1112 -- the constructors for expr
1113 app : expr -> expr -> expr
1114 lam : type -> (expr -> expr) -> expr
1115 tapp : expr -> type -> expr
1116 tlam : kind -> (type -> expr) -> expr

```

Figure 6. The higher-order abstract syntax description of System F that the generator consumes. Sorts are declared as Type, and a binder appears as a function argument, so `lam` binds one `expr` in its body. From this the generator emits the syntax of fig. 1 and the rule set of section 3.3

stay defined by recursion over the vector, so their clauses compete with the rules exactly as the function model's do, and opacity is needed all the same.

Nothing is lost on the side of provability, and the supplement carries the experiment. With maps as vectors every law of section 3.3 is a theorem under `--safe`, the four η -laws included. Equality of vectors is equality of data, so that variant assumes nothing, where ours postulates function extensionality. We keep functions as the more intuitive model. The laws are theorems either way, so we can report that the η -laws are never once applied by hand across the POPLmark development.

The designs differ on cost. Every vector operation recurses on the scope, so laws the function model closes by `refl` become inductions the generator has to emit. Making the map a tuple by recursion on the scope instead buys the η -laws definitionally, from Agda's η -rule for records, and forfeits the rules, because the operations then reduce on their own and every rule mentioning one overlaps its defining clauses unjoinably. The trade is made per definition. Transparency buys η and forbids rules. Opacity buys rules and forbids η . We take opacity, and drop η for the independent reason of section 3.2.

Vectors remain the right choice where functions are not available, notably when formalizing type theory in type theory [7], and they are what the uniform renaming/substitution treatments [6, 25] of section 5 use.

The original σ -calculus treats its operations purely syntactically, describing lookup itself through instantiation. The de Bruijn algebra then supplied a semantic model in functions, which made the rules provable and usable for automated propositional reasoning. This work does both at once. The model proves the laws, and the syntactic reading makes them apply.

4.3 POPLmark and POPLmark Reloaded

To test the construction at a scale where substitution reasoning hurts, we solved the POPLmark Challenge, Parts 1A, 1B,

2A, 2B and 3, and POPLmark Reloaded including its sums extension, on top of generated cores. The development is roughly 9,100 lines of Agda, 6,400 of them excluding blanks and comments, of which 2,800 are generated infrastructure and 3,500 hand-written metatheory. It spans five languages, $F_{<}$, $F_{<}$ with records, $F_{<}$ with pattern matching, STLC and STLC with sums, and `fun-ext` is its only postulate.

The point of comparison is the substitution reasoning a de Bruijn development normally pays for. In the Rocq solutions by the AUTOSUBST authors the σ -laws are applied by an `asimpl` tactic, invoked explicitly wherever substitutions must be normalized. Here there is no such step, because normalization happens during conversion. Section 5 places the coverage among published solutions. The claim rests on three measurements, each a property of the whole development rather than of a chosen example.

- The η -laws, the ones we cannot register, are applied by hand zero times. Their absence is not felt.
- No proof uses a `rewrite` clause. Thirty transports remain across the metatheory modules, of which exactly two are along a substitution equation, and both are the same lemma.
- That lemma, which we call `[]-as-ren`, is a single law, and its failure is not an accident of the tooling. It says

$$t[(x)_0] = t[x \cdot^R \text{id}^R]^R$$

and the two sides are two distinct normal forms. The left one unfolds to $t[(x) \cdot^S \text{id}^S]^S$, which is extension-shaped, while coincidence needs a syntactic $\langle \xi \rangle$ to collapse into mode V. The rule that would bridge them is $(x) \cdot^S \langle \xi \rangle \rightarrow \langle x \cdot^R \xi \rangle$, in the same $S \rightarrow R$ direction as the rest of the coercion group. Registering it costs four non-joinable pairs, and the one that survives every attempt to complete the system is

$$(x[\xi \uparrow^R s]^R)[(y) \cdot^S \langle \xi_1 \rangle]^S$$

whose reducts meet only if `lift-consR` can fire, which needs composition at a variable to fold. It pushes,

because folding there overlaps $x[\text{wk}^R s]^R = \text{succ } x$ unjoinably. The one hand-proved substitution fact is therefore the push-at-V decision seen from the substitution side, not an independent gap, and we supply the missing join as an induction on the term.

The system leaves two costs in place. First, a function defined by recursion over the syntax does not commute with substitution for free. The size and shape measures the Challenge needs still take their own induction, because with an abstract argument the reduction laws do not fire. Second, a rewrite system constrains what may be matched as well as what may be proved. An earlier version of our transitivity proof for subtyping recursed on $Q[\xi]^R$ to make the induction structural in Q . It type checks in the ordinary sense and is then rejected by `--local-confluence-check`. Matching on Q puts $(\forall \dots)[\xi]^R$ into the clause's left-hand side, where it overlaps a reduction law unjoinably.

5 Related Work

Three lines of work meet here. Solutions to the POPLmark Challenge fix the benchmark our evaluation is measured against, generic and generated syntax libraries are what we would replace, and a small recent literature attacks the same tension between provability and definitional behaviour from the semantic side.

POPLmark solutions. Fifteen solutions to the Challenge have been published. Of those, four ever attempted the records extension, three the pattern-matching extension, and four Part 3. Berghofer's Isabelle/HOL de Bruijn development is the only one covering Parts 1, 2 and 3 together, and it animates the congruence-rule presentation because its code generator cannot execute the challenge's evaluation contexts. We cover 1A, 1B, 2A and 2B, and answer Part 3 as a decision procedure for the evaluation-context relation, machine-checking all seven tests of the challenge's own `step.poplmark`, at the cost of running on 2B without the pattern `let`. On POPLmark Reloaded we join Allais's generic-syntax Agda solution, Beluga and the two Rocq AUTOSUBST solutions as a complete STLC and STLC+ solution. Allais additionally proves completeness of the inductive characterization of strong normalization, which we do not. The nearest neighbours in technique are Schäfer and Stark's Rocq de Bruijn solutions, by the authors of AUTOSUBST and of the Reloaded benchmark. Coverage is what makes the claim about the substitution reasoning underneath it testable.

We have discussed AUTOSUBST and related research [32, 35, 36], user-defined rewrite rules [13, 14, 23] and opaque definitions [18] throughout this work.

In recent work on local rewrite rules [24], the authors also rewrite σ -calculus rules with support for first-class renamings in Rocq. They too model renamings and substitutions as functions, but cannot rewrite all laws against that model,

because with local rewrite rules the instantiation of the equations must hold definitionally rather than propositionally to maintain consistency. Rocq also has global rewrite rules [23], and we expect our approach to translate. Rocq, however, cannot instantiate rewrite rules with proofs, so the consistency guarantee would be lost.

Kaposi and Pujet obtain a definitional substitution calculus by a semantic route [21], presenting the initial category with families so that substitutions are metatheoretical functions inside strict presheaves over the syntax of a dependent type theory. Composition of substitutions is then composition of functions, so the laws hold by conversion and most of the transports that an indexed-inductive presentation forces disappear. Their strictified syntax has the induction principle of the initial model, which a shallow embedding of the same syntax does not. In their formalization, ten of the eleven equations of the strictified category with families hold by reflexivity. The eleventh is surjective pairing for substitutions. We reach that same law from the syntactic side, where inductive variables destroy its left-hand side. Neither development makes surjective pairing definitional. They derive lifting as $(\gamma \circ p, q)$ instead of taking it as primitive, so $\text{id}^S \uparrow^S s = \text{id}^S$ fails for them, for the reason section 3.2 gives. We keep lifting primitive and keep that law. They also obtain the law for the empty context, which we cannot register, because the statement that any two substitutions out of the empty scope agree has no left-hand side to match. Their strictness comes from the presentation rather than from rewriting, but rewrite rules still carry their encoding of observational type theory and of the quotient inductive-inductive syntax, and those rules are postulated rather than proved. We prove every rule before registering it, and we establish confluence and termination for the result.

Chen, Nordvall Forsberg and Tsai [12] give an intrinsic representation of type theory in Cubical Agda as quotient inductive-inductive-recursive types, and obtain a syntax that needs no transports, no postulates and no custom rewrite rules. Where they replace the rewrite rules by a quotient, we keep the plain inductive type and pay for it with a confluence obligation.

Generators for binder boilerplate predate and accompany AUTOSUBST. Needle & Knot [22] generates the boilerplate for complex binding forms, including the relational lemmas that AUTOSUBST leaves out, but does not aim at an equational theory of substitution and so has no equations to turn into rules. Strongly typed term representations in Rocq [9] are the ancestor of the scoped syntax we use.

Another line of research concerns generic programming approaches [4, 5] that build syntax from general building blocks supporting sums, products and binders, and thereby abstract over renamings and substitutions. The approach carries over to a scoped, multi-sorted setting [29]. We believe generic programming and rewriting the σ -calculus are

orthogonal. Applying our rule set to the universe of syntaxes with binding would remove the need for external code generation altogether.

The ABSTRACT BINDING TREES library for Agda shares a goal with AUTOSUBST and is used, for instance, in Siek's work [34]. It builds an object language from a generic syntax, but supports only extrinsic scoping and single-sorted syntax, and focuses on generic theorems about substitution. Its experimental support for rewriting with σ -calculus laws misses rules and does not pass the confluence check.

Wadler experiments with explicit substitutions and implements weakening as a constructor, thereby avoiding renamings [39]. He rewrites some of the σ -calculus laws but does not pass the confluence checker. Section 1 discussed what that representation costs.

Wadler and colleagues also investigate a uniform treatment of renamings and substitutions in a simply typed setting [6], originating in work by McBride [25]. It unifies repeated similar lemmas, merging all four compositionality lemmas into one, by hiding the structural dependence of renamings and substitutions in a uniform, *Mode*-indexed vector, and lets the termination checker see that dependence where it matters. Saffrich [29] instead hides it behind instance resolution, in a scoped setting where the uniform structure is again a function, and adds reflection-based code generation for the syntax-dependent lemmas, which could be adopted here in place of our external generator.

Adapting our approach to a setting where renamings and substitutions are treated uniformly is a logical next step, but the equational theory would have to be extended in non-obvious ways to support uniform proofs over both. It would also require a two-level rewrite system, one on the type level, normalizing expressions in the lattice of *Modes* generated by $\mathbb{V} \sqsubseteq \mathbb{T}$ [6], and one for the adapted σ -calculus. In our experiments the interaction of the two layers made confluence hard to preserve.

6 Future Work

Opaque Definitions for Rewriting

The construction suggests a general recipe for integrating normalization for suitable algebraic structures into a proof assistant: define operations for the structure; establish a confluent rewrite system over them; instantiate the structure with an underlying model and prove the laws inside *opaque* blocks; and, with the model hidden, *REWRITE* the equations.

Classical sets are one candidate. Reasoning about them in Agda usually means lists up to permutation together with a uniqueness predicate, which is tedious and generates a great deal of uninteresting code. Their characteristic function supplies the model the recipe asks for. Sets become $\mathcal{S} = \mathcal{A} \rightarrow \text{Bool}$ for some $\mathcal{A} : \text{Set } \ell$, the operations become opaque functions

opaque

$\emptyset : \mathcal{S}$

$\emptyset a = \text{false}$

$_ \cup _ : \mathcal{S} \rightarrow \mathcal{S} \rightarrow \mathcal{S}$

$(A \cup B) a = A a \vee B a$

and laws such as $(A \cup \emptyset) \equiv A$ become provable and registerable. The boolean ring interpretation [20] is a natural starting point for the rule set, but it needs rewriting modulo associativity and commutativity. Dedukti [8] supports this and Agda does not, which is the missing ingredient.

Implications for Intrinsically Typed Syntax

The construction applies as it stands to intrinsically typed syntax whose type index lives in *Set*. Richer syntax, where a substitution appears in the type index, is out of its reach for now, and that is where the phenomenon known as transport hell is worst. The transfer lemma (*subst* in Agda) then has to carry every step of every proof. Intrinsically typed System F has been formalized in Agda before, executably and in full [10], and a binary logical relation over it [30] is a close example of the cost. Type-in-type and expression-in-expression reasoning stays easy, but type-in-expression substitution also affects the index of the expression substituted into, and the authors spend thousands of lines on the resulting transports. Rewriting the σ -calculus rules on the syntax of the typing index should remove them, and should generalize to other intrinsically typed formalizations.

7 Conclusion

We have turned the normalization procedure of the σ -calculus into definitional equalities in Agda. The operations must compute, so that the laws are provable, and stay inert, so that the rules match. Opaque definitions give both.

The theory we register is $\sigma_{\uparrow}$ instantiated once per mode, with the rules that first-class renamings force between the two. Which laws survive is forced, because native inductive variables and surjective pairing cannot coexist in one rewrite system. Agda certifies local confluence and AProVE termination, a generator emits it from a syntax description, and the POPLmark challenges exercise it at scale.

References

- [1] M. Abadi, L. Cardelli, P.-L. Curien, and J.-J. Levy. 1989. Explicit substitutions. In *Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Francisco, California, USA) (POPL '90). Association for Computing Machinery, New York, NY, USA, 31–46. doi:10.1145/96709.96712
- [2] Andreas Abel and Brigitte Pientka. 2011. Higher-Order Dynamic Pattern Unification for Dependent Types and Records. In *Typed Lambda Calculi and Applications*, Luke Ong (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 10–26.
- [3] Robin Adams. 2004. Formalized metatheory with terms represented by an indexed family of types. In *Proceedings of the 2004 International Conference on Types for Proofs and Programs* (Jouy-en-Josas, France)

- (*TYPES'04*). Springer-Verlag, Berlin, Heidelberg, 1–16. doi:10.1007/11617990_1
- [4] Guillaume Allais, Robert Atkey, James Chapman, Conor McBride, and James McKinna. 2021. A Type and Scope Safe Universe of Syntaxes with Binding: Their Semantics and Proofs. arXiv:2001.11001 [cs.PL] <https://arxiv.org/abs/2001.11001>
- [5] Guillaume Allais, James Chapman, Conor McBride, and James McKinna. 2017. Type-and-scope safe programs and their proofs. In *Proceedings of the 6th ACM SIGPLAN Conference on Certified Programs and Proofs* (Paris, France) (*CPP 2017*). Association for Computing Machinery, New York, NY, USA, 195–207. doi:10.1145/3018610.3018613
- [6] Thorsten Altenkirch, Nathaniel Burke, and Philip Wadler. 2025. Substitution without copy and paste. <https://nottingham-repository.worktribe.com/output/51884690/substitution-without-copy-and-paste>. In *Logical Frameworks and Meta-Languages: Theory and Practice (LFMTP 25)* (2025-07-19). ACM, Birmingham, UK, 17 pages.
- [7] Thorsten Altenkirch and Ambrus Kaposi. 2016. Type theory in type theory using quotient inductive types. *SIGPLAN Not.* 51, 1 (Jan. 2016), 18–29. doi:10.1145/2914770.2837638
- [8] Ali Assaf, Guillaume Burel, Raphaël Cauderlier, David Delahaye, Gilles Dowek, Catherine Dubois, Frédéric Gilbert, Pierre Halmagrand, Olivier Hermant, and Ronan Saillard. 2023. Dedukti: a Logical Framework based on the $\lambda\Pi$ -Calculus Modulo Theory. arXiv:2311.07185 [cs.LO] <https://arxiv.org/abs/2311.07185>
- [9] Nick Benton, Chung-Kil Hur, Andrew Kennedy, and Conor McBride. 2012. Strongly Typed Term Representations in Coq. *Journal of Automated Reasoning - JAR* 49 (08 2012), 1–19. doi:10.1007/s10817-011-9219-0
- [10] James Chapman, Roman Kireev, Chad Nester, and Philip Wadler. 2019. System F in Agda, for Fun and Profit. In *Mathematics of Program Construction: 13th International Conference, MPC 2019, Porto, Portugal, October 7–9, 2019, Proceedings* (Porto, Portugal). Springer-Verlag, Berlin, Heidelberg, 255–297. doi:10.1007/978-3-030-33636-3_10
- [11] Arthur Charguéraud. 2012. The Locally Nameless Representation. *Journal of Automated Reasoning* 49, 3 (2012), 363–408. doi:10.1007/s10817-011-9225-2
- [12] Liang-Ting Chen, Fredrik Nordvall Forsberg, and Tzu-Chun Tsai. 2026. Can We Formalise Type Theory Intrinsically without Any Compromise? A Case Study in Cubical Agda. In *Proceedings of the 15th ACM SIGPLAN International Conference on Certified Programs and Proofs* (Rennes, France) (*CPP '26*). Association for Computing Machinery, New York, NY, USA, 201–215. doi:10.1145/3779031.3779090
- [13] Jesper Cockx. 2020. Type Theory Unchained: Extending Agda with User-Defined Rewrite Rules. In *25th International Conference on Types for Proofs and Programs (TYPES 2019) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 175)*, Marc Bezem and Assia Mahboubi (Eds.). Schloss Dagstuhl -- Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 2:1–2:27. doi:10.4230/LIPIcs.TYPES.2019.2
- [14] Jesper Cockx, Nicolas Tabareau, and Théo Winterhalter. 2021. The taming of the rew: a type theory with computational assumptions. *Proc. ACM Program. Lang.* 5, POPL, Article 60 (Jan. 2021), 29 pages. doi:10.1145/3434341
- [15] Pierre-Louis Curien, Thérèse Hardin, and Jean-Jacques Lévy. 1996. Confluence properties of weak and strong calculi of explicit substitutions. *J. ACM* 43, 2 (March 1996), 362–397. doi:10.1145/226643.226675
- [16] N. G. de Bruijn. 1972. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem. *Indagationes Mathematicae (Proceedings)* 75, 5 (1972), 381–392. doi:10.1016/1385-7258(72)90034-0
- [17] Jean-Yves Girard. 1972. *Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur*. PhD thesis. Éditeur inconnu. <https://www.cs.cmu.edu/~kw/scans/girard72thesis.pdf>
- [18] Daniel Gratzer, Jonathan Sterling, Carlo Angiuli, Thierry Coquand, and Lars Birkedal. 2025. Controlling unfolding in type theory. *Mathematical Structures in Computer Science* 35 (2025), e38, 1–37. doi:10.1017/S0960129525100327
- [19] Thérèse Hardin, Luc Maranget, and Bruno Pagano. 1998. Functional runtime systems within the lambda-sigma calculus. *Journal of Functional Programming* 8, 2 (1998), 131–176. doi:10.1017/S0956796898003001
- [20] Jieh Hsiang. 1985. Refutational theorem proving using term-rewriting systems. *Artificial Intelligence* 25, 3 (1985), 255–300. doi:10.1016/0004-3702(85)90074-8
- [21] Ambrus Kaposi and Loïc Pujet. 2025. Type Theory in Type Theory using a Strictified Syntax. *Proc. ACM Program. Lang.* 9, ICFP, Article 266 (Aug. 2025), 31 pages. doi:10.1145/3747535
- [22] Steven Keuchel, Stephanie Weirich, and Tom Schrijvers. 2016. Needle & Knot: Binder Boilerplate Tied Up. In *Proceedings of the 25th European Symposium on Programming Languages and Systems - Volume 9632*. Springer-Verlag, Berlin, Heidelberg, 419–445. doi:10.1007/978-3-662-49498-1_17
- [23] Yann Leray, Gaëtan Gilbert, Nicolas Tabareau, and Théo Winterhalter. 2024. The Rewster: Type Preserving Rewrite Rules for the Coq Proof Assistant. In *15th International Conference on Interactive Theorem Proving (ITP 2024) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 309)*, Yves Bertot, Temur Kutsia, and Michael Norrish (Eds.). Schloss Dagstuhl -- Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 26:1–26:18. doi:10.4230/LIPIcs.ITP.2024.26
- [24] Yann Leray and Théo Winterhalter. 2026. Encode the Cake and Eat It Too: Controlling Computation in Type Theory, Locally. *Proc. ACM Program. Lang.* 10, POPL, Article 62 (Jan. 2026), 30 pages. doi:10.1145/3776704
- [25] Conor McBride. 2005. Type-preserving renaming and substitution. (2005). <http://strictlypositive.org/ren-sub.pdf> Unpublished manuscript.
- [26] Gerald E. Peterson and Mark E. Stickel. 1981. Complete Sets of Reductions for Some Equational Theories. *J. ACM* 28, 2, 233–264. doi:10.1145/322248.322251
- [27] Frank Pfenning and Conal Elliott. 1988. Higher-order abstract syntax. In *Proceedings of the ACM SIGPLAN 1988 Conference on Programming Language Design and Implementation (PLDI '88)*. Association for Computing Machinery, New York, NY, USA, 199–208. doi:10.1145/53990.54010
- [28] John C. Reynolds. 1974. Towards a theory of type structure. In *Programming Symposium*, B. Robinet (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 408–425. <https://www.cs.cmu.edu/afs/cs/user/jcr/ftp/theotypestr.pdf>
- [29] Hannes Saffrich. 2024. Abstractions for Multi-Sorted Substitutions. In *15th International Conference on Interactive Theorem Proving (ITP 2024) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 309)*, Yves Bertot, Temur Kutsia, and Michael Norrish (Eds.). Schloss Dagstuhl -- Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 32:1–32:19. doi:10.4230/LIPIcs.ITP.2024.32
- [30] Hannes Saffrich, Peter Thiemann, and Marius Weidner. 2024. Intrinsically Typed Syntax, a Logical Relation, and the Scourge of the Transfer Lemma. In *Proceedings of the 9th ACM SIGPLAN International Workshop on Type-Driven Development (Milan, Italy) (TyDe 2024)*. Association for Computing Machinery, New York, NY, USA, 2–15. doi:10.1145/3678000.3678201
- [31] Steven Schäfer, Gert Smolka, and Tobias Tebbi. 2015. Completeness and Decidability of de Bruijn Substitution Algebra in Coq. In *Proceedings of the 2015 Conference on Certified Programs and Proofs* (Mumbai, India) (*CPP '15*). Association for Computing Machinery, New York, NY, USA, 67–73. doi:10.1145/2676724.2693163

- [32] Steven Schäfer, Tobias Tebbi, and Gert Smolka. 2015. Autosubst: Reasoning with de Bruijn Terms and Parallel Substitutions. In *Interactive Theorem Proving*, Christian Urban and Xingyuan Zhang (Eds.). Springer International Publishing, Cham, 359--374. https://www.ps.uni-saarland.de/Publications/documents/SchaeferEtAl_2015_Autosubst_-Reasoning.pdf
- [33] Kiraku Shintani and Nao Hirokawa. 2024. Compositional Confluence Criteria. *Logical Methods in Computer Science* Volume 20, Issue 1 (2024), 6:1--6:33. doi:10.46298/lmcs-20(1:6)2024
- [34] Jeremy G. Siek and Tianyu Chen. 2021. Parameterized cast calculi and reusable meta-theory for gradually typed lambda calculi. *Journal of Functional Programming* 31 (2021), e30. doi:10.1017/S0956796821000241
- [35] Kathrin Stark. 2020. *Mechanising Syntax with Binders in Coq*. Ph.D. Dissertation. Saarland University, Saarbrücken, Germany. https://www.ps.uni-saarland.de/Publications/documents/Stark_2020_Mechanising.pdf Dissertation zur Erlangung des Grades des Doktors der Ingenieurwissenschaften (Dr.-Ing.).
- [36] Kathrin Stark, Steven Schäfer, and Jonas Kaiser. 2019. Autosubst 2: reasoning with multi-sorted de Bruijn terms and vector substitutions. In *Proceedings of the 8th ACM SIGPLAN International Conference on Certified Programs and Proofs* (Cascais, Portugal) (CPP 2019). Association for Computing Machinery, New York, NY, USA, 166--180. doi:10.1145/3293880.3294101
- [37] Christian Urban, Stefan Berghofer, and Cezary Kaliszyk. 2013. Nominal 2. *Archive of Formal Proofs* 2013 (February 2013), 1 pages. <https://isa-afp.org/entries/Nominal2.html>, Formal proof development.
- [38] Christian Urban and Christine Tasson. 2005. Nominal techniques in Isabelle/HOL. In *Proceedings of the 20th International Conference on Automated Deduction* (Tallinn, Estonia) (CADE' 20). Springer-Verlag, Berlin, Heidelberg, 38--53. doi:10.1007/11532231_4
- [39] Philip Wadler. 2024. Explicit Weakening. *Electronic Proceedings in Theoretical Computer Science* 413 (Nov. 2024), 15--26. doi:10.4204/eptcs.413.2